NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 11: Understanding Program Efficiency II

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 11 at a glance

Lecture 31

Hidden work in Python
Scans, copies, shifts, index construction, and amortized growth.

Lecture 32

Recursive growth
Total calls, local work, live frames, and cached subproblems.

Lecture 33

Scaling experiments and engineering choices
A complete moving-window case: count, verify, benchmark, and interpret.

Expose the work → Compare growth → Measure fairly → Preserve the result

ENG 101 Course Specification, Session 11; user-provided Weeks 10–12 outline.

Course scope and continuity

Week 10 established

- Input size and selected operations.
- Big-O and tight growth.
- Characteristic basic classes.

Week 11 develops

- Hidden work and composite tasks.
- Recursive time and memory.
- Scaling evidence and trade-offs.

Course boundary

Week 12 remains the dedicated searching-and-sorting implementation unit. These examples add no assignment deadlines, grading weights, or new submission rules.

ENG 101 Course Specification, Session 11; user-provided Weeks 10–12 outline.

NAZARBAYEV UNIVERSITY

LECTURE 31

Hidden Work in Python Programs

Look inside operations before assigning a complexity class

NAZARBAYEV UNIVERSITY

Lecture 31 learning goals

1. Identify hidden scans, copies, and shifts in familiar Python code.
2. Keep separate input sizes visible instead of calling everything n.
3. Explain amortized list growth and average-case dictionary lookup.
4. Improve repeated work while preserving order, duplicates, and units.
5. Include preprocessing, output storage, and validation in the stated scope.

Central question

What work must this operation perform, even when it occupies one line?

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Warm-up: two loops do not settle the answer

```
# A fixed inner workload.
for i in range(n):
    for j in range(3):
        count += 1
```

```
# A growing inner workload.
for i in range(n):
    for j in range(i):
        count += 1
```

Predict before running

With each counter initialized to zero, give its final value for $n = 4$, then a formula for general n.

Individual then pairs - 3 min

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Write the analysis contract first

Part	Question	Example
Input size	What can grow?	q installed IDs; m approved IDs
Selected work	What do we count?	ID equality comparisons
Input case	Which arrangement?	Every installed ID absent
Output	What must be returned?	One Boolean per installed ID
Cost model	What is treated as bounded?	Short-string comparison; ordinary hashing

Scope matters

If constructing a lookup table is necessary, state whether that preparation is included.

Wang, Appendix G.3–G.6, G.12 and G.32; Week 10 recap.

NAZARBAYEV UNIVERSITY

Familiar operations hide different work

Existing Python list	Typical cost	Reason
`len(a)`, `a[i]`	$O(1)$	Stored length; direct position
`sum(a)`, `x in a`	$O(n)$ worst case	May inspect every element
`a[:k]`, `a.copy()`	$O(k)$; $O(n)$	Copy references into a new list
`a.insert(0, x)`	$O(n)$	Shift existing references
`a.append(x)`	$O(1)$ amortized	Occasional capacity growth

Declared teaching model

Existing lists; bounded-cost numeric or element operations. A custom comparison may do additional work.

Wang, Appendix G.15; Python Design FAQ and CPython list implementation.

NAZARBAYEV UNIVERSITY

A short expression can repeatedly copy a prefix

```
def build_by_concatenation(values):
    """Double each value; preserve order and the
     input list."""
    result = []
    for value in values:
        # This creates a new list and recopies the
     growing prefix.
        result = result + [2 * value]
    return result
```

Contract

Return doubled values in the same order; leave the input unchanged.

Predict

For `[2, 3, 4]`, what is returned? Which old values are recopied?

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Add the work done by every concatenation

Pass	Result length	Slots populated
1	1	1
2	2	2
3	3	3
4	4	4

$$C(n) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} = \Theta(n^2).$$

Do not confuse time and peak storage

Quadratic cumulative copying does not mean all earlier lists remain alive at once. Peak list storage is linear here.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Append preserves the result with less repeated work

```
def build_by_append(values):
    """Return the same doubled values without
     repeated prefix copies."""
    result = []
    for value in values:
        # Append one new value to the existing
     output list.
        result.append(2 * value)
    return result
```

```
[4, 6, 8]
```

What changed?

The growing prefix is no longer rebuilt on every pass.

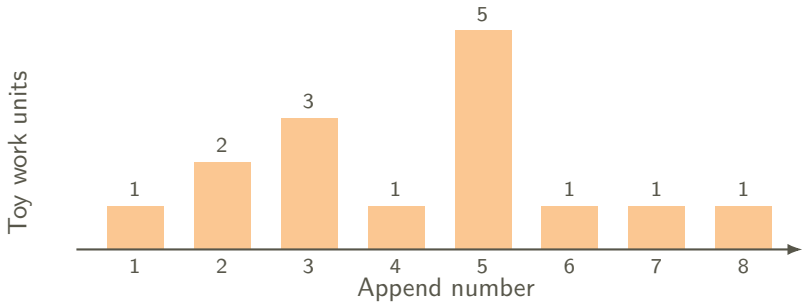
What did not change?

Values, ordering, output length, and non-mutation.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Amortized cost looks at a whole sequence



Illustrative doubling-capacity model

One new-value write per append; resizing copies existing entries. Total work for 8 appends: $8 + (1 + 2 + 4) = 15$.

The capacities and bar heights are a teaching model, not CPython timing or its exact allocation rule.

NAZARBAYEV UNIVERSITY

Average-case and amortized are different claims

Average-case

Average over a specified distribution of inputs or relevant randomized behavior.

Amortized

Bound total work across a sequence of operations, including occasional expensive steps.

Use the right qualification

List append: amortized $O(1)$. Dictionary key lookup: average/expected $O(1)$ under suitable hashing assumptions, not a universal worst-case guarantee.

Wang, Appendix G.12–G.15; Python Design FAQ.

Removing the first element repeats shifts

```
def drain_front(values):
    """Sum a COPY by removing its first
     item; use small integers here."""
    work = values.copy()  # Preserve the
     caller's list.
    total = 0
    while work:
        # Removing index 0 shifts the
     remaining references left.
        total += work.pop(0)
    return total
```

```
def drain_back(values):
    """Sum a COPY by removing its last item
     ; use small integers here."""
    work = values.copy()
    total = 0
    while work:
        # No later elements must be shifted
      after removing the last item.
        total += work.pop()
    return total
```

$$\text{Front shifts: } (n-1) + (n-2) + \cdots + 0 = \frac{n(n-1)}{2}.$$

Both functions copy the input first. Small integer sums agree; floating-point summation order may differ.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

One visible loop; a second scan inside

```
def approval_flags_list(installed, approved):
    """One Boolean per installed ID, retaining its
     order and duplicates."""
    flags = []
    for sensor_id in installed:
        # With a list, this membership test may scan
      all approved IDs.
        flags.append(sensor_id in approved)
    return flags
```

Two independent sizes

$q =$ `len(installed)`
$m =$ `len(approved)`

Preserve meaning

Return one flag for each installed record, including duplicate IDs.

Wang, Appendix G.14.9 and G.31; adaptation returning every approval flag.

NAZARBAYEV UNIVERSITY

Derive the hidden membership cost

Scenario	Comparisons	Output
All IDs absent	qm	Still returns all q flags
Every match at first position	q for $m \geq 1$	Still returns all q flags
No installed IDs	0 comparisons	Empty result list
Equal growing sizes	$q = m = n$	Worst-case quadratic time

Include the loop even with an empty approved list

For $m = 0$, membership comparisons are zero but creating q flags still costs $\Theta(q)$. A useful worst-case expression is $\Theta(q(m + 1))$ for $q \geq 1$.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Build a dictionary index once

```
def build_approval_lookup(approved):
    """Build a membership index for short, hashable
     string identifiers."""
    lookup = {}
    for sensor_id in approved:
        # Repeated approved IDs need only one entry
     for membership.
        lookup[sensor_id] = True
    return lookup
```

Familiar Week 6 structure

Keys represent approved IDs. Each value is merely a membership marker.

Preprocessing cost

Average/expected $O(m)$ construction and $O(u)$ entries, where $u \leq m$ is the number of distinct IDs.

Wang, Appendix G.15 and G.31; dictionary adaptation of the membership-index idea.

NAZARBAYEV UNIVERSITY

Reuse the index for the batch

```
def approval_flags_lookup(installed, lookup):
    """Return flags using an ALREADY BUILT
     dictionary index."""
    flags = []
    for sensor_id in installed:
        # Membership checks dictionary keys, not
     dictionary values.
        flags.append(sensor_id in lookup)
    return flags
```

```
[True, False, True]
```

Whole first batch

Expected work: $O(m + q)$.
Extra lookup entries: $O(u)$.
Required flag output: $O(q)$.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Do not rebuild the index for each query

Plan for B batches of q IDs	Expected or worst-case model
Scan an approved list each time	$O(Bq(m+1))$ worst case
Rebuild a dictionary for each batch	$O(B(m+q))$ expected
Build once; reuse for all batches	$O(m+Bq)$ expected

An index is only as current as its source

Rebuild or update the index when the approved IDs change. Reuse must not create stale engineering decisions.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Test the contract after changing the structure

```
# Order and repeated installed IDs are part of the requirement.
installed = ["T-2", "T-9", "T-2"]
approved = ["T-2", "T-2", "T-1"]
original = installed.copy()
lookup = build_approval_lookup(approved)
assert approval_flags_lookup(installed, lookup) == [True, False, True]
assert installed == original  # Optimization must not mutate the input.
```

Add more cases

Empty lists, one match, no matches, all matches, duplicate approved IDs, and a changed approved collection.

Week 7 testing practices; original checks for the G.31-inspired adaptation.

A method call does not announce its cost

```
class ReadingLog:
    """An existing nonempty numeric record; units
     belong to its contract."""
    def __init__(self, readings):
        if not readings:
            raise ValueError("at least one reading
     is required")
        self.readings = readings.copy()  #
     Construction costs O(n).
    def latest(self):
        # One positional lookup on an existing list.
        return self.readings[-1]
    def mean(self):
        # A short method call can hide a complete
     scan.
        return sum(self.readings) / len(self.
     readings)
```

After construction

`log.latest()`: one list access.
`log.mean()`: a complete sum.

Construction is not free

The initializer copies n references.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Repeated invariant work can dominate

```
def centre_repeated(readings):
    """Subtract the same mean repeatedly;
     intentionally inefficient."""
    centred = []
    for value in readings:
        # The unchanged input is scanned again on
     every pass.
        mean = sum(readings) / len(readings)
        centred.append(value - mean)
    return centred
```

Read before measuring

The input is not changed inside this function. Does its mean change from one output item to the next?

Count

n sum calls, each visiting n readings: $\Theta(n^2)$ work.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Compute the unchanged mean once

```
def centre_once(readings):
    """Move an invariant calculation outside the
     output loop."""
    if not readings:
        return []
    mean = sum(readings) / len(readings)
    centred = []
    for value in readings:
        # The mean is valid while the readings
     remain unchanged.
        centred.append(value - mean)
    return centred
```

```
[-2.0, 0.0, 2.0]
[-2.0, 0.0, 2.0]
```

Two scans add

One sum plus one output pass: $\Theta(n)$ total work. Required output remains $\Theta(n)$.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Some nested work sums to a linear total

```
def geometric_work(n):
    """Count visits for full, half, quarter, ...
     workloads; n >= 0."""
    visits = 0
    size = n
    while size > 0:
        for _ in range(size):
            visits += 1  # One selected visit, not
     one machine instruction.
        size //= 2
    return visits
```

For $n = 8$

$8 + 4 + 2 + 1 = 15$ visits.

Geometric sum

$n + \lfloor n/2 \rfloor + \cdots < 2n$ for $n \geq 1$.
Therefore $\Theta(n)$ work.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Activity: same stages, different local workload

```
def full_scan_each_stage(n):
    """Count n visits at each halving stage; n >= 1.
     """
    visits = 0
    size = n
    while size > 1:
        for _ in range(n):
            visits += 1  # Work stays n even as the
     stage counter shrinks.
        size //= 2
    return visits
```

Predict for $n = 8$

How many stages? How many visits at each stage? How many visits in total?

Pairs - 4 min

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

Output is part of the computational requirement

Return all corrected readings

n output values must be produced and stored in a returned list.

Return only a warning count

One scalar may suffice, but this is a different requested result.

Do not claim an improvement by deleting the requirement

Compare implementations with the same values, order, multiplicity, units, and documented error behavior.

Wang, Appendix G.16, G.21 and G.32; engineering-contract synthesis.

NAZARBAYEV UNIVERSITY

A hidden-work review checklist

1. State the growing inputs and the required output.
2. Expand scans, slices, copies, shifts, and method calls.
3. Add sequential stages; sum dependent inner workloads.
4. Qualify hashing and amortized operations.
5. Count preparation and retained output separately.
6. Rerun known-answer, boundary, and non-mutation tests.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

Lecture 31 exit ticket

Three short explanations

1. Why can a list-membership expression inside one loop be quadratic?
2. What does amortized constant append cost NOT promise?
3. For q flags and m approved IDs, compare list lookup with a dictionary built once.

Individual - 3 min

Next lecture

The number of active recursive frames is not the number of calls made.

Wang, Appendix G.13–G.16 and G.31; original teaching adaptations.

NAZARBAYEV UNIVERSITY

LECTURE 32

Recursive Growth and Memory

Count local work, all calls, and the largest live state separately

NAZARBAYEV UNIVERSITY

Lecture 32 learning goals

1. Distinguish total calls from maximum recursion depth.
2. Account for copying work inside a recursive call.
3. Explain one-half, two-half, and overlapping subproblems.
4. Compare simple recursion, iteration, and a small explicit cache.
5. State time and auxiliary-space claims with their assumptions.

Central question

How much work is done at each node, and how many nodes are visited?

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Warm-up: recursion is a form, not a class

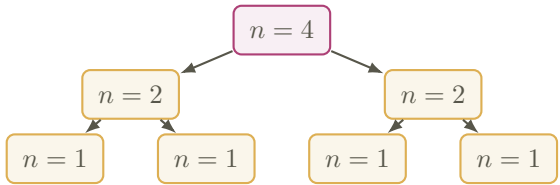
Pattern	Question to answer
One call on n - 1	How many steps to reach the base case?
One call on floor(n / 2)	How quickly does the input shrink?
Two calls on smaller inputs	Are they disjoint or overlapping?
A slice before every call	How much data is copied locally?

Predict one growth pattern - 3 min

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Total nodes and depth answer different questions



7 total calls
3 frames on a deepest path

Count the resource you mean

Time depends on work across all visited nodes. Stack storage depends on the deepest active call chain.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

One item per call, without a new list

```
def sum_by_index(values, index=0):
    """Sum a short list recursively without copying
    tails; start at 0."""
    if index == len(values):
        return 0
    # Pass the same list and a new position to the
    smaller subproblem.
    return values[index] + sum_by_index(values,
    index + 1)
```

Cost model

One indexed value and one addition per non-base call.

For n values

$n + 1$ calls; n additions.
$T(n) = T(n-1) + O(1)$.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

One item per call, but with a new tail

```
def sum_by_slicing(values):
    """Sum a short list recursively; each tail slice
     allocates a list."""
    if not values:
        return 0
    # Copies len(values)-1 references before the
     next call.
    return values[0] + sum_by_slicing(values[1:])
```

The hidden operation

`values[1:]` copies a tail into a new list before the recursive call.

Same selected arithmetic

Still n additions and n+1 calls. The copied references add extra work.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Trace the copied tails for four elements

Active call input length	Next slice length	Cumulative copies
4	3	3
3	2	5
2	1	6
1	0	6
0	No recursive call	6

$$C_{\text{copy}}(n) = (n-1) + (n-2) + \cdots + 0 = \frac{n(n-1)}{2}.$$

Total time

$T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$, despite only one recursive call per level.

Wang, Appendix C 14.11, C.16 and C.22; Guttag, Ch. 9. Original analyses

NAZARBAYEV UNIVERSITY

Which lists remain live at the base case?

Original frame holds 4 input references

Next frame holds a copied tail of 3

Next frame holds a tail of 2

Next frame holds a tail of 1

For this implementation

Waiting frames retain their list arguments. The copied tails can coexist: $\Theta(n^2)$ additional live references, plus $\Theta(n)$ frames.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Same mathematical sum, different resource use

Implementation	Time model	Auxiliary space
Recursive index	$\Theta(n)$	$\Theta(n)$ frames
Recursive tail slice	$\Theta(n^2)$	$\Theta(n^2)$ retained references
Iterative subtotal	$\Theta(n)$	$\Theta(1)$ scalar state

```
def sum_iteratively(values):
    """Sum once using a running scalar, without copying the input."""
    total = 0
    for value in values:
        total += value  # One arithmetic update per element.
    return total
```

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

A halving recurrence with constant local work

```
def halving_depth(n):
    """Return the number of floor-halvings to reach
     1; integer n >= 1."""
    if n <= 1:
        return 0
    # One recursive call on approximately half the
     previous size.
    return 1 + halving_depth(n // 2)
```

For $n = 16$

$16 \to 8 \to 4 \to 2 \to 1$: four reductions, five calls.

Recurrence

$T(n) = T(\lfloor n/2 \rfloor) + \Theta(1)$
Time and frames grow as $\Theta(\log n)$ for large n.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Logarithmic calls can still copy linear data

```
def first_by_half_slicing(values):
    """A deliberately wasteful first-item retrieval
    for cost analysis."""
    if not values:
        raise ValueError("a nonempty list is
    required")
    if len(values) == 1:
        return values[0]
    # Few calls do not eliminate the cost of copying
      each half-list.
    return first_by_half_slicing(values[:len(values)
      // 2])
```

Power-of-two length n

Copied tails:
$n/2 + n/4 + \cdots + 1 = n - 1.$

Not a recommended algorithm

`values[0]` already solves first-item retrieval directly.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

One half is not the same as two halves

Follow one branch

$T(n) = T(n/2) + c$

One node per level; $\Theta(\log n)$ total work.

Process both halves

$T(n) = 2T(n/2) + c$

A complete tree; $\Theta(n)$ total work.

Assumptions

Positive power-of-two n for the exact tree picture; constant work per node and no copying.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Balanced summation visits both subintervals

```
def balanced_sum(values, left=0, right=None):
    """Sum by dividing an index interval; no list slices are created."""
    if right is None:
        right = len(values)
    if left == right:
        return 0  # Empty interval.
    if right - left == 1:
        return values[left]  # One leaf value.
    middle = (left + right) // 2
    # Two disjoint subproblems, then one addition at this node.
    return (balanced_sum(values, left, middle)
            + balanced_sum(values, middle, right))
```

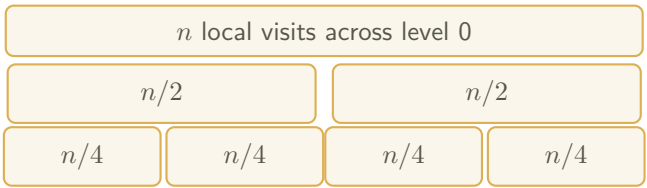
Index intervals avoid slice allocations

n leaf values, $n - 1$ internal additions, $2n - 1$ calls for $n \geq 1$. The deepest stack contains $\lceil \log_2 n \rceil + 1$ frames.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

A scan at every level changes the class



Each level totals n. There are $\log_2 n$ internal levels.

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

Conceptual full-scan tree; sorting implementations are reserved for Week 12.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Recursive patterns worth explaining

Recursive pattern	Local work	Time growth
One call on $n-1$	Constant	$\Theta(n)$
One call on $n/2$	Constant	$\Theta(\log n)$
Two calls on $n/2$	Constant	$\Theta(n)$
Two calls on $n/2$	Linear in current size	$\Theta(n \log n)$
One call on $n-1$	Linear-size tail copy	$\Theta(n^2)$

Explain, do not memorize

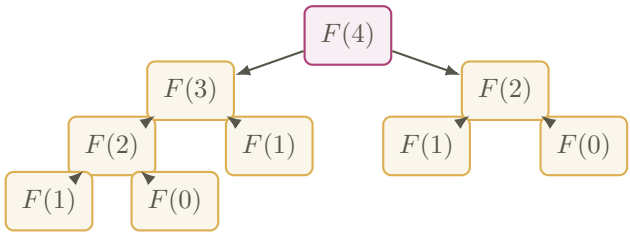
Specify the base case, subproblem sizes, number of calls, and work outside those calls.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Week 6 Fibonacci: overlapping subproblems

$$F(0) = F(1) = 1, \qquad F(n) = F(n-1) + F(n-2).$$



Nine calls produce the single value $F(4) = 5$. Repeated nodes are repeated computation.

Week 6 convention; Wang, Appendix G.14.11; complete instructor-drawn call tree.

NAZARBAYEV UNIVERSITY

Count Fibonacci calls, not just depth

```
def fibonacci_counted(n):
    """Return (F(n), calls), with F(0)=F(1)=1; use
     small n >= 0."""
    if n <= 1:
        return 1, 1  # This base-case call counts
     too.
    left, left_calls = fibonacci_counted(n - 1)
    right, right_calls = fibonacci_counted(n - 2)
    return left + right, 1 + left_calls +
     right_calls
```

n	F(n)	Calls
5	8	15
10	89	177
20	10,946	21,891

Growth

Exponential time; $O(2^n)$ is an upper bound. Linear-depth stack.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Iteration keeps the needed history, not the tree

```
def fibonacci_iterative(n):
    """Compute the Week 6 sequence for a nonnegative
     integer n."""
    if n <= 1:
        return 1
    previous, current = 1, 1
    for _ in range(2, n + 1):
        # Store only two sequence values; their bit
     lengths can grow.
        previous, current = current, previous +
     current
    return current
```

For n = 20

Same value: 10,946.
Only 19 recurrence additions.

Model distinction

Two retained sequence values, not two fixed-size bits or bytes.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

A dictionary can remember a solved subproblem

Without memory

Calling F(4) can lead to the same F(2) subtree more than once.

With a correct cache

Store a completed value under n. When the same n is requested again, return it.

```
# A fresh cache uses the same base cases as our recurrence.
cache = {0: 1, 1: 1}
answer = fibonacci_cached(10, cache)
print(answer, len(cache))  # 89 and 11 entries after this call.
```

Scope

A small caching example connects dictionaries to efficiency; a full dynamic-programming course is not required.

Wang, Appendix G.22.2; Guttag, Section 13.1 as supplementary reading. Original small example.

NAZARBAYEV UNIVERSITY

Read the cache before making recursive calls

```
def fibonacci_cached(n, cache):
    """Use a correct cache initially containing {0:
     1, 1: 1}; n >= 0."""
    if n in cache:
        return cache[n]  # A previously solved
     subproblem.
    # Store the result once, then reuse it in later
     calls.
    cache[n] = (fibonacci_cached(n - 1, cache)
                + fibonacci_cached(n - 2, cache))
    return cache[n]
```

```
89
Cached entries: 11
Reused result: 89
```

Fresh-cache growth

Expected $\Theta(n)$ arithmetic/hash operations, $\Theta(n)$ cached entries, and $\Theta(n)$ stack frames.

Wang, Appendix G.22.2; explicit memoization teaching adaptation.

NAZARBAYEV UNIVERSITY

A cache trades computation for retained state

Method	Work model	Retained state
Naive Fibonacci	Exponential total calls	Linear-depth call stack
Fresh-cache Fibonacci	Expected $\Theta(n)$	Linear cache and stack
Iterative Fibonacci	$\Theta(n)$ arithmetic	Two sequence values

Choose for the actual task

For one sequence value, iteration is simpler here. Caching becomes useful when solved subproblems are reused in a broader calculation.

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

A reused value must still mean the same thing

Engineering example

A cached stress depends on force, area, the unit convention, and the model. Caching only by force can reuse the wrong result after area changes.

- Use a key that identifies all relevant inputs.
- Do not hide changing global parameters behind an incomplete key.
- Specify when cached values must be cleared or rebuilt.
- Do not use a shared mutable default dictionary accidentally.

Saved computation is useful only when the saved result is still valid

Wang, Appendix G.22 and G.32; Weeks 4, 6 and 9 state-contract connections.

NAZARBAYEV UNIVERSITY

Activity: defend the time and space claims

Program	Local work	Your time/space explanation
Index-recursive sum	One value and one recursive call	...
Tail-slice recursive sum	One slice and one recursive call	...
Balanced index sum	Two disjoint calls; one addition	...
Fresh-cache Fibonacci	Reuse previously solved states	...

Pairs - 4 min

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

Practical depth and size are still limited

Mathematical termination is not enough

A recursion may reach its base case in theory but exceed the permitted Python call depth first.

- Trace with tiny inputs and keep recursive demonstrations small.
- Count maximum active frames separately from all completed calls.
- Do not raise the recursion limit to hide a poor data-processing design.
- Prefer an iterative form when a long sequential update is sufficient.

Integer arithmetic caveat

Large Fibonacci values and factorials require more bits; our simple arithmetic-operation counts are not full bit-complexity analyses.

Python sys.getrecursionlimit documentation; Wang, Appendix G.16 and G.21.

Lecture 32 exit ticket

1. Why can one recursive call per level still produce quadratic time?
2. Why does a logarithmic-depth tree not necessarily take logarithmic total time?
3. What work is excluded when timing a cache hit after precomputation?
4. Name one accuracy or contract check required after replacing recursion with iteration.

Individual - 3 min

Wang, Appendix G.14.11, G.16 and G.22; Guttag, Ch. 9. Original analyses.

NAZARBAYEV UNIVERSITY

LECTURE 33

Scaling Experiments and Engineering Choices

Connect a growth argument to repeatable measurements and a valid result

NAZARBAYEV UNIVERSITY

Lecture 33 learning goals

1. Use doubling ratios without treating them as proof.
2. Analyze a complete moving-window calculation in n and w.
3. Compare two implementations with the same output contract.
4. Run and interpret a fair local scaling experiment.
5. Explain workload, memory, accuracy, and end-to-end trade-offs.

Central question

Does the proposed improvement remain correct, and what evidence supports its benefit?

Wang, Appendix G.23–G.25 and G.32; original experiment and worked examples.

NAZARBAYEV UNIVERSITY

Predict doubling before timing

Pure leading work model	C(2n) / C(n)
1	1
$\log_2 n$	$1 + 1/\log_2 n$
n	2
$n \log_2 n$	$2(1 + 1/\log_2 n)$
n^2	4
n^3	8
2^n	2^n

These ratios describe the displayed functions, not every function sharing an $O(\cdot)$ upper bound.

Wang, Appendix G.11 and G.24; exact ratios for declared mathematical models.

NAZARBAYEV UNIVERSITY

A measured ratio is a diagnostic, not a theorem

$$R(n) = \frac{t(2n)}{t(n)}, \qquad \widehat{p}(n) = \frac{\log R(n)}{\log 2}.$$

For a pure power law

If $t(n) = an^p$, the ratio is 2^p and this estimate recovers p.

For real measurements

Overhead, input cases, cache behavior, allocations, and background work can change the ratio.

Keep two kinds of evidence

A code-based operation argument explains growth. Timing shows behavior over the sizes and machine actually tested.

Wang, Appendix G.24–G.25; instructor interpretation of empirical order.

NAZARBAYEV UNIVERSITY

Engineering case: every full-window mean

$$\overline{T}_i = \frac{T_i + T_{i+1} + \cdots + T_{i+w-1}}{w}, \quad i = 0, \ldots, n - w.$$

Readings (deg C)	Window width	Required means (deg C)
[20,22,24,26,28]	3	[22,24,26]

Contract

Return all n - w + 1 means in time order; no padding; no input modification. Require integer width between 1 and n and finite numeric readings.

Instructor-designed engineering case within Session 11; builds on Weeks 5 and 10.

NAZARBAYEV UNIVERSITY

Direct implementation: sum every window

```
def moving_mean_direct(values, width):
    """Return every full-window mean; input is not modified."""
    check_width(values, width)
    means = []
    for start in range(len(values) - width + 1):
        total = 0.0
        for index in range(start, start + width):
            total += values[index]  # Revisit overlapping readings.
        means.append(total / width)
    return means
```

Read the nested work

Adjacent windows overlap, but each window is summed from the beginning again. The same readings are revisited.

Original moving-window example; exact code in examples/week11_models.py.

NAZARBAYEV UNIVERSITY

The right count keeps both dimensions

$$C_{\text{direct}}(n, w) = w(n - w + 1), \qquad 1 \le w \le n.$$

How width scales	Work interpretation
Fixed w as n grows	$\Theta(n)$
$w = n/4$ (n divisible by 4)	$\Theta(n^2)$
$w = n$	One full-window mean: linear work
$w = 1$	One output per input: linear work

Count definition

Each addition into a window subtotal is counted once. Divisions, loop control, checks, and output writes are additional work.

Original exact-count derivation; Wang, Appendix G.14 and G.20.

NAZARBAYEV UNIVERSITY

Reuse the overlap between adjacent windows

$$S_{i+1} = S_i - T_i + T_{i+w}.$$



$S_0 = 66$

$S_1 = 72$

$66 - 20 + 26 = 72$; mean $= 72/3 = 24$

One removal, one addition

After building the first window, do not rescan the shared interior.

Original diagram and algebraic moving-window update.

NAZARBAYEV UNIVERSITY

Running implementation: update the subtotal

```
def moving_mean_running(values, width):
    """Reuse a window total; reassess floating-point error for real data."""
    check_width(values, width)
    total = 0.0
    for index in range(width):
        total += values[index]  # Build the first window only once.
    means = [total / width]
    for index in range(width, len(values)):
        total -= values[index - width]  # Remove the outgoing value.
        total += values[index]          # Add the incoming value.
        means.append(total / width)
    return means
```

Original moving-window implementation; comments explain the two phases.

NAZARBAYEV UNIVERSITY

Compare like-for-like arithmetic counts

$$C_{\text{running}}(n, w) = w + 2(n - w) = 2n - w.$$

n	w = n/4	Direct additions	Running + / -
8	2	14	14
16	4	52	28
32	8	200	56
64	16	784	112

Do not label count ratios as measured speedups

These count selected arithmetic updates. Both methods also create and return the same number of outputs.

Original exact counts; checked with instrumented index accesses.

NAZARBAYEV UNIVERSITY

Test outputs, widths, and preserved data

```
# A hand-checkable record and an independent expected answer.
values = [20.0, 22.0, 24.0, 26.0, 28.0]
original = values.copy()
for method in [moving_mean_direct, moving_mean_running]:
    assert method(values, 3) == [22.0, 24.0, 26.0]
    assert method(values, 1) == values  # Smallest valid width.
    assert method(values, 5) == [24.0]  # One full-length window.
    assert values == original          # No input mutation.
```

Invalid-width tests

Width 0, width greater than n, noninteger width, Boolean width, and an empty input must be rejected by the documented checks.

Week 7 testing practice; original tests for the two moving-mean methods.

NAZARBAYEV UNIVERSITY

Numerical accuracy can defeat an elegant update

```
# A deliberate stress test for floating-point cancellation.
values = [1e16, 1.0, 1.0, 1.0]
print(moving_mean_direct(values, 2))
print(moving_mean_running(values, 2))
```

Observed with ordinary binary floats

Direct: `[5e15, 1.0, 1.0]`
Running: `[5e15, 0.5, 0.5]`

Do not accept a faster wrong result

Reassess the algorithm, numerical representation, or recomputation strategy when the error violates the application requirement.

Original executed counterexample; Python floating-point documentation for background.

NAZARBAYEV UNIVERSITY

Work and stored output remain separate

Resource	Direct windows	Running subtotal
Arithmetic updates	$w(n-w+1)$	$2n-w$
Extra scalar workspace	Constant	Constant
Returned means	$n-w+1$	$n-w+1$
Input record	Same input; preserved	Same input; preserved

What the improvement buys

Less repeated calculation, not fewer required output values. In both implementations, returned-list storage scales with the number of means.

Original resource analysis; Wang, Appendix G.16 and G.21.

NAZARBAYEV UNIVERSITY

Design a fair scaling experiment

1. Use the same finite input and width for both implementations.
2. Check agreement before timing; retain a copy to detect mutation.
3. Choose several sizes; here $w = n/4$ deliberately grows with n.
4. Exclude fixture construction, counters, printing, and file I/O.
5. Repeat each case; divide each batch time by its number of calls.
6. Record the runtime environment and every raw trial.

Recorded statistic

The following slides use the fastest of seven batch-normalized trials, with all trials retained in the source package.

Wang, Appendix G.23–G.25; official Python timeit documentation.

A repeatable timing call

```
from timeit import repeat
# Prepare the data before entering the timed callable.
values = [20.0 + (i % 21) * 0.5 for i in range(256)]
width = 64
def task():
    return moving_mean_running(values, width)
# Five trials, each containing 100 calls of the same task.
trials = repeat(task, repeat=5, number=100)
print(min(trials) / 100)  # Fastest observed seconds per call.
```

This short demonstration uses five trials; the recorded scaling dataset uses seven and adaptive batch lengths.

Python timeit.repeat documentation; complete local protocol in tools/generate_data.py.

NAZARBAYEV UNIVERSITY

Recorded timings: the scaling is visible

n	Direct (μs)	Running (μs)	Direct ratio	Running ratio
128	67.8	7.08	–	–
256	243	13.9	3.58	1.97
512	1.14e+03	30.8	4.69	2.21
1024	4.92e+03	65.5	4.33	2.13
2048	2.01e+04	137	4.08	2.09

One local experiment

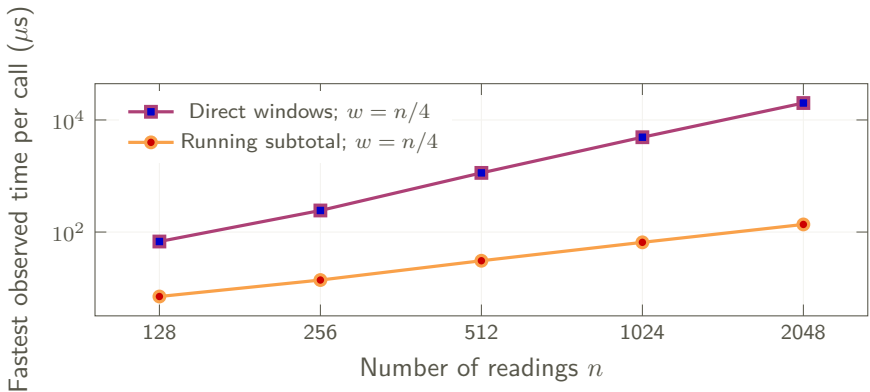
CPython; width n/4; fastest of seven trials. Ratios compare adjacent sizes. The raw timings and environment record are included.

Question

Which pattern is consistent with approximately quadratic growth? Which is closer to linear?

NAZARBAYEV UNIVERSITY

Plot the measured work time, not an invented curve



Both axes are logarithmic. Connecting lines guide the eye; these are measured points, not fitted

NAZARBAYEV UNIVERSITY

Interpret a measurement without overstating it

Supported conclusion

For this workload and tested sizes, the running method used substantially less measured time, consistent with its lower arithmetic count.

Not established

Universal speedups, real-time deadlines, numerical reliability on every dataset, or a proof from timing alone.

Changing w changes the experiment

With width fixed, the direct method is also linear in n. A comparison without reporting width is incomplete.

Wang, Appendix G.24–G.25 and G.32; interpretation of the original benchmark.

NAZARBAYEV UNIVERSITY

Use a declared operation budget carefully

$$n = 2000, \quad w = 500, \quad M = n - w + 1 = 1501.$$

$$C_{\text{direct}} = 750\,500, \qquad C_{\text{running}} = 3500.$$

Illustrative budget: 100,000 arithmetic updates

The running method fits this selected-operation budget; the direct method does not.

Not a wall-clock guarantee

A count budget ignores implementation constants, input/output overhead, and the accuracy requirement. It is an explicit model, not a measured deadline.

Original hypothetical budget and exact-count calculation.

NAZARBAYEV UNIVERSITY

More sensors multiply the workload

m independent channels

For each channel, process n readings using the same width w. Channels retain separate records.

Plan	Arithmetic-count model
Direct windows on each channel	$m\,w(n-w+1)$
Running subtotal on each channel	$m(2n-w)$
All results retained	$m(n-w+1)$ output values

Define all growing dimensions

If $m = n$ and $w = n/4$, direct work is cubic; running work is quadratic. Required output storage is still quadratic.

Wang, Appendix G.19–G.21; original multichannel extension.

NAZARBAYEV UNIVERSITY

Speeding one stage does not speed everything equally

Hypothetical stage	Before	After
Read and validate data	0.60 s	0.60 s
Calculation	0.30 s	0.03 s
Write report	0.10 s	0.10 s
End-to-end total	1.00 s	0.73 s

Ten times faster calculation; only 1.37 times overall

Total speedup $= 1.00/0.73 \approx 1.37$. Improve the stage that limits the actual task.

Wang, Appendix G.26 and G.32; original hypothetical pipeline arithmetic.

NAZARBAYEV UNIVERSITY

Peer review: is the proposed improvement acceptable?

Proposal

A programmer keeps only the last moving mean, removes the width checks, and reports a large speedup.

1. Which output requirement changed?
2. Which previously documented failures may now go unnoticed?
3. Which comparison would be fair instead?
4. What numerical stress test should be retained?

Pairs - 4 min

Wang, Appendix G.23–G.25 and G.32; original experiment and worked examples.

NAZARBAYEV UNIVERSITY

Laboratory 4 preparation: analysis plus evidence

Practice with the supplied case

- Write the n,w cost model.
- Run the known-answer checks.
- Reproduce a small scaling table.
- Explain a time–space–accuracy trade-off.

Check the final Moodle brief

- Task wording and marking criteria.
- Permitted assistance and collaboration.
- Submission format and deadline.

Laboratory 4 is listed in the course specification. This preparation slide does not create new graded requirements.

ENG 101 Course Specification, Session 11 and E-Learning section.

A defensible efficiency report

1. What result is required, with which units and accuracy?
2. Which input dimensions grow, and what input case is analyzed?
3. What exact work or asymptotic bound is justified?
4. What preparation, memory, and output costs are included?
5. Which tests still pass after the change?
6. What was measured, on which environment, and with what limitations?

A faster program is useful only when it still solves the specified problem

Wang, Appendix G.32–G.33; course-wide engineering evidence synthesis.

NAZARBAYEV UNIVERSITY

Lecture 33 exit ticket and Week 12 bridge

For n = 12 readings and width w = 3

1. How many full-window means must be returned?
2. How many selected additions does the direct method perform?
3. How many + / - updates does the running method perform?
4. Why would timing alone not establish the better engineering choice?

Individual - 3 min

Next: searching and sorting

Apply the same cost discipline to linear search, bisection search, and sorting algorithms.

User-provided Weeks 11–12 outline; original exit-ticket counts.

NAZARBAYEV UNIVERSITY

Reference: familiar container costs

Operation	Cost model	Important qualification
`a[i]`, `len(a)`	$O(1)$	Existing list
`a[:k]`, `a.copy()`	$O(k)$; $O(n)$	Shallow reference copies
`x in a`	$O(n)$ worst case	List; equality may have a cost
`a.append(x)`	$O(1)$ amortized	An individual resize costs more
`k in d`, `d[k]`	$O(1)$ average/expected	Dictionary; ordinary hashing
`a.insert(0, x)`	$O(n)$	Shifts later references

These are implementation-aware teaching models, not universal complexity promises for arbitrary user-defined objects.

Wang, Appendix G.15; Python Design FAQ and CPython list implementation.

Optional: justify the amortized resize model

Copied references at resizes: $1 + 2 + 4 + \cdots + 2^k < 2n$.

New-value writes: $n \implies$ total modeled work $< 3n$.

Average per append in this toy strategy

Less than 3 selected work units over the whole sequence; hence amortized $O(1)$.

The distinction matters

The model doubles capacity for explanation. CPython uses its own proportional overallocation policy, not this exact doubling schedule.

Wang, Appendix G.13; original geometric-capacity proof; CPython implementation clarification.

NAZARBAYEV UNIVERSITY

Optional: factorial growth is about all orderings

```
from itertools import permutations
from math import factorial
# Small example: visit every ordering of four distinct labels.
visits = 0
for order in permutations(range(4)):
    visits += 1
print(visits)        # 24 orderings visited.
print(factorial(4))  # One integer result with the same value.
```

Separate the contracts

$n!$ tuples exist. Explicitly storing every length-n tuple requires $\Theta(n\,n!)$ slots, not just one integer.

Wang, Appendix G.10.7; original itertools demonstration.

NAZARBAYEV UNIVERSITY

Optional: include preparation before repeated searches

Scan each time

For q lookups in n unsorted records, worst-case work is modeled by qn.

Prepare an ordered index

In a simplified unit-coefficient model: $n \log_2 n + q \log_2 n$.

A model comparison, not a universal threshold

For $n > \log_2 n$, the second expression is smaller when $q > n \log_2 n/(n - \log_2 n)$. Constants and data updates change a real crossover.

Search and sort implementations, ordering assumptions, and update costs are developed in Week 12.

Wang, Appendix G.28; original unit-coefficient comparison; Week 12 preview.

NAZARBAYEV UNIVERSITY

Reference: measure without changing the workload

1. Use a fresh output on every timed call.
2. Reset mutable inputs or caches when measuring cold-start behavior.
3. Keep counters and plotting out of the timed region.
4. Retain raw trial vectors as well as the selected summary.
5. Check the effect of garbage collection for allocation-heavy applications.
6. Use a representative end-to-end experiment before making deployment claims.

Reproduce this package

Run `python tools/generate_data.py` explicitly to replace the recorded data. Compiling LaTeX alone does not rerun the benchmark.

Python timeit documentation; source-package reproducibility instructions.

Week 11 quick vocabulary

Term	Meaning in this unit
Input size	The independently growing dimensions of the task
Local work	Work done by one call, excluding its child calls
Recursion depth	Length of a longest active call chain
Auxiliary space	Working storage apart from supplied input and reported output
Amortized cost	Total sequence cost distributed across its operations
Cache / memoization	Stored results reused for the same subproblem
Empirical order	A finite-size estimate from measured scaling

Synthesis of Wang, Appendix G.12–G.25 and the examples in this deck.

NAZARBAYEV UNIVERSITY

Course sources and teaching adaptations

- **Scope:** ENG 101 Course Specification, Session 11: complexity classes and characteristic examples; Laboratory 4. CLOs 1 and 2.
- **Engineering reference:** Y. Wang, *Programming for Engineers with Python*, supplied v20260815 manuscript, Appendix G.11–G.16, G.21–G.25, G.28 and G.31–G.33.
- **Algorithm reference:** J. V. Guttag, *Introduction to Computation and Programming Using Python*, 2nd ed., Ch. 9; Section 13.1 for the optional caching connection.

Adaptations are explicit

Dictionary approval flags, counted recursive patterns, moving-window means, diagrams, and local timings are newly constructed teaching examples. No textbook timing is presented as a new measurement.

Supplied course materials; detailed alignment in SOURCE_MAP.md.

Technical references and what comes next

Official Python sources

Design FAQ: list representation, dictionary hashing, floating-point background.
timeit: repeatable microbenchmarks and limitations.
sys.getrecursionlimit: interpreter recursion depth.
CPython list implementation: overallocation and list operations.

Week 12

Searches and sorts: requirements, implementations, traces, and costs.

Keep the engineering discipline

Define the task, preserve its meaning, justify growth, and test the result.

Primary technical documentation checked 14 September 2026; user-provided continuation.

Week 11 in one sentence

A short program can hide a large computation.

Expose the work, justify the growth,
measure the right task, and keep the result trustworthy.

Understanding Program Efficiency II